LING285-L4

Course: LING285: Human Language and Technology

Title of Lecture: Lecture 4 - Dynamical Systems 2

Date: 9/3/2026

Professor: Khalil Iskarous

Section 1: The Shortest Path Problem

- The **shortest path problem** asks for the cheapest route through a network of connected points, from a starting point to an ending point. Each point is a **state**, which is a place we can be at a given moment. Each arrow is a possible move from one state to another, and each arrow has a number attached to it called its **cost**. The cost can stand for distance, time, effort, or anything we want to keep as small as possible. The total cost of a route is the sum of the costs on every arrow along it, and the goal is to find the route whose total is smallest.
- In our network, we start at **A** and want to reach **J**. The states are arranged in columns called **stages**, and every arrow moves us forward exactly one stage. We can never skip a stage or move backward.
    - Stage one contains only A. Stage two contains B and C. Stage three contains D, E, and F. Stage four contains G, H, and I. Stage five contains only J, the goal.
    - Any route from A to J therefore crosses exactly four arrows and makes exactly four decisions, one at each stage.
- The costs out of A and out of the second stage are the first ones we face.
    - From A, the arrow to B costs 12 and the arrow to C costs 7.
    - From B, the arrow to D costs 5, the arrow to E costs 6, and the arrow to F costs 9.
    - From C, the arrow to D costs 14, the arrow to E costs 10, and the arrow to F costs 11.
- The costs out of the third stage lead into the fourth stage.
    - From D, the arrow to G costs 8, the arrow to H costs 7, and the arrow to I costs 10.
    - From E, the arrow to G costs 9, the arrow to H costs 7, and the arrow to I costs 9.
    - From F, the arrow to G costs 10, the arrow to H costs 7, and the arrow to I costs 8.
- The fourth stage has only one arrow out of each state, and each of those arrows goes to J.
    - From G to J costs 5, from H to J costs 9, and from I to J costs 8.
- This network is a model of **decision-making over time**. Every choice has an immediate cost, and every choice also controls which states, and therefore which future costs, are available at the next stage. A choice that looks good now can lock us into expensive moves later.

Section 2: The Greedy Algorithm

- A **greedy algorithm** makes each decision by looking only at the arrows leaving the current state and taking the cheapest one. It never looks further ahead than one move. At every state it asks only which next step is cheapest right now.
- Following the greedy rule from A through the network produces one specific route.
    - At A, the choices are 12 to B and 7 to C. The cheaper move is 7, so greed takes us to **C**.
    - At C, the choices are 14 to D, 10 to E, and 11 to F. The cheapest move is 10, so greed takes us to **E**.
    - At E, the choices are 9 to G, 7 to H, and 9 to I. The cheapest move is 7, so greed takes us to **H**.
    - At H, there is only one arrow, which goes to J and costs 9, so we are forced to take it.
- The total cost of the greedy route A, C, E, H, J is:

$$7 + 10 + 7 + 9 = 33$$

- This is not the best we can do. The route A, B, D, G, J starts with the more expensive first move of 12, but its total cost is smaller:

$$12 + 5 + 8 + 5 = 30$$

- **Greed is not good** for this problem. The greedy rule failed at the very first decision, since the cheap move to C led into a part of the network where every continuation was more costly. Greed also walked right into H, which ends with the large final cost of 9. The greedy algorithm has no way of knowing about costs that come later, so a late obstacle can make an early cheap-looking choice a bad one.

Section 3: A Small Example of Greed Failing

- A very small network makes the failure of greed easy to see. From a starting state there are two routes to an ending state, and each route passes through one middle state.
  - On the top route, the first arrow costs 10 and the second arrow, from the top middle state to the end, also costs 10.
  - On the bottom route, the first arrow costs only 2, but the second arrow, from the bottom middle state to the end, costs 100.
- A greedy decision at the start compares 10 with 2 and picks the bottom route. It then gets stuck paying 100 on the next move. The two totals are:

$$10 + 10 = 20 \qquad 2 + 100 = 102$$

- The top route is five times cheaper than the greedy one, even though its first step was five times more expensive.
- Written inside each middle state is the cost of getting from that state to the end: 10 inside the top middle state and 100 inside the bottom middle state. Once each middle state carries this number, the decision at the start becomes simple. We add the cost of the first arrow to the number stored in the state it leads to, and compare 20 with 102. Storing the cost of the rest of the journey inside each state removes the blindness of greed.

Section 4: The Exhaustive Algorithm

- An **exhaustive algorithm** solves the shortest path problem by brute force. It lists every possible route from start to end, adds up the cost of each one, and picks the route with the smallest total. Nothing is skipped, so the answer is **guaranteed to be correct**. The method is very accurate.
- In our network, the number of routes comes from multiplying the number of choices at each stage.
  - From A there are 2 choices, from the second stage there are 3 choices, and from the third stage there are 3 choices. From the fourth stage there is only 1 choice, since every state there goes straight to J.
  - The number of complete routes is:

$$2 \times 3 \times 3 \times 1 = 18$$

- Each of these 18 routes crosses 4 arrows, so each one needs 3 additions to get its total. Checking every route takes 54 additions, followed by comparing 18 totals to find the smallest.
- The problem with exhaustive search is its growth. Every added stage multiplies the number of routes by the number of choices at that stage, and every added state per stage makes each of those multipliers larger. If there are **many stages and many states**, the number of routes explodes, and checking all of them takes a very, very long time. Exhaustive search is accurate but becomes impractical for large problems, while greedy search is fast but can be wrong. We need a method that is both accurate and efficient.

Section 5: Bellman's Idea — Imagining the Values

- **Bellman** approached the problem by starting with an imaginary step. Imagine we already knew the best possible cost of getting from B to J, and the best possible cost of getting from C to J. We call these numbers V_{BJ} and V_{CJ}. The letter V stands for **Value**, and the Value of a state is the lowest total cost of getting from that state to the goal.

- If those two Values were known, the best cost from A to J, which is V_{AJ}, would follow from one short calculation.
    - Any route out of A must go first to either B or C, since those are the only two arrows leaving A.
    - If we go to B, we pay 12 for that arrow, and then the best we can possibly do for the rest of the trip is V_{BJ}. So the best total through B is $12 + V_{BJ}$.
    - If we go to C, we pay 7 for that arrow, and then the best we can do for the rest of the trip is V_{CJ}. So the best total through C is $7 + V_{CJ}$.
    - The best cost from A is whichever of these two totals is smaller. This is written as:

$$V_{AJ} = \min \left\{ \begin{array}{l} 12 + V_{BJ} \\ 7 + V_{CJ} \end{array} \right\}$$

- The word **min** means we take the minimum, which is the smallest number in the set inside the braces. Each number in the set is built the same way. It is the cost of one arrow plus the Value of the state that arrow leads to.
- The difficulty is that we **don't know** V_{BJ} or V_{CJ}. The equation for V_{AJ} is correct, but it depends on two numbers we have not yet found.
- Bellman's general strategy handles exactly this situation. **To solve a problem, decompose it into subproblems you can solve.** The finding of V_{AJ} becomes two smaller problems, finding V_{BJ} and finding V_{CJ}. The phrase "if you can" matters here. If a subproblem still cannot be solved directly, it gets broken down again into even smaller subproblems.

Section 6: Breaking the Problem into Subproblems

- Bellman's reasoning at A applies in the same way at B. We don't know the best cost from B to J, but if we knew the best costs from D, E, and F to J, then we could find the best cost from B to J.
    - From B we can go to D for 5, to E for 6, or to F for 9. After each move, the best we can do is the Value of the state we arrived at. The best cost from B is the smallest of the three totals:

$$V_{BJ} = \min \left\{ \begin{array}{l} 5 + V_{DJ} \\ 6 + V_{EJ} \\ 9 + V_{FJ} \end{array} \right\}$$

- C works the same way, with the arrows to D costing 14, to E costing 10, and to F costing 11:

$$V_{CJ} = \min \left\{ \begin{array}{l} 14 + V_{DJ} \\ 10 + V_{EJ} \\ 11 + V_{FJ} \end{array} \right\}$$

- These equations still depend on unknown Values, so the decomposition continues one more stage. The Values of D, E, and F depend on the Values of G, H, and I.
    - From D, the arrows cost 8 to G, 7 to H, and 10 to I:

$$V_{DJ} = \min \left\{ \begin{array}{l} 8 + V_{GJ} \\ 7 + V_{HJ} \\ 10 + V_{IJ} \end{array} \right\}$$

- From E, the arrows cost 9 to G, 7 to H, and 9 to I:

$$V_{EJ} = \min \left\{ \begin{array}{l} 9 + V_{GJ} \\ 7 + V_{HJ} \\ 9 + V_{IJ} \end{array} \right\}$$

- From F, the arrows cost 10 to G, 7 to H, and 8 to I:

$$V_{FJ} = \min \left\{ \begin{array}{l} 10 + V_{GJ} \\ 7 + V_{HJ} \\ 8 + V_{IJ} \end{array} \right\}$$

- At the fourth stage, the subproblems finally become ones we can solve directly. Each of G, H, and I has exactly one arrow, and it goes straight to J. There is no decision to make, so each Value is simply the cost of that single arrow:

$$V_{GJ} = 5 \qquad V_{HJ} = 9 \qquad V_{IJ} = 8$$

- The equations form a chain of dependence that runs from the end of the network back to the start. The known Values of G, H, and I feed into the equations for D, E, and F. Once D, E, and F are known, their Values feed into the equations for B and C. Once B and C are known, their Values feed into the single equation for A. Information about the cost of reaching J travels **backward** through the network, one stage at a time, starting from the states closest to the goal.

[CONTINUE]

Section 7: Calculating the Values, Then Going Greedy

- With the Values of G, H, and I known, the Values of the third stage can be calculated directly by substituting $V_{GJ} = 5$, $V_{HJ} = 9$, and $V_{IJ} = 8$ into each equation.
  - For D, the three totals are 8 plus 5, 7 plus 9, and 10 plus 8:

$$V_{DJ} = \min \left\{ \begin{array}{l} 8+5 \\ 7+9 \\ 10+8 \end{array} \right\} = \min \left\{ \begin{array}{l} 13 \\ 16 \\ 18 \end{array} \right\} = 13$$

- For E, the three totals are 9 plus 5, 7 plus 9, and 9 plus 8:

$$V_{EJ} = \min \left\{ \begin{array}{l} 9+5 \\ 7+9 \\ 9+8 \end{array} \right\} = \min \left\{ \begin{array}{l} 14 \\ 16 \\ 17 \end{array} \right\} = 14$$

- For F, the three totals are 10 plus 5, 7 plus 9, and 8 plus 8:

$$V_{FJ} = \min \left\{ \begin{array}{l} 10+5 \\ 7+9 \\ 8+8 \end{array} \right\} = \min \left\{ \begin{array}{l} 15 \\ 16 \\ 16 \end{array} \right\} = 15$$

- In all three cases the smallest total comes from going to G. G has the smallest Value of the fourth stage, 5, and that outweighs the differences in the arrow costs.
- The third-stage Values, $V_{DJ} = 13$, $V_{EJ} = 14$, and $V_{FJ} = 15$, now go into the equations for B and C.
  - For B, the totals are 5 plus 13, 6 plus 14, and 9 plus 15:

$$V_{BJ} = \min \left\{ \begin{array}{l} 5+13 \\ 6+14 \\ 9+15 \end{array} \right\} = \min \left\{ \begin{array}{l} 18 \\ 20 \\ 24 \end{array} \right\} = 18$$

- For C, the totals are 14 plus 13, 10 plus 14, and 11 plus 15:

$$V_{CJ} = \min \left\{ \begin{array}{l} 14+13 \\ 10+14 \\ 11+15 \end{array} \right\} = \min \left\{ \begin{array}{l} 27 \\ 24 \\ 26 \end{array} \right\} = 24$$

- Finally, $V_{BJ} = 18$ and $V_{CJ} = 24$ go into the equation for A:

$$V_{AJ} = \min \left\{ \begin{array}{l} 12+18 \\ 7+24 \end{array} \right\} = \min \left\{ \begin{array}{l} 30 \\ 31 \end{array} \right\} = 30$$

- The best possible cost from A to J is **30**. Every state in the network now carries its Value: A is 30, B is 18, C is 24, D is 13, E is 14, F is 15, G is 5, H is 9, and I is 8.
- The best path comes from **going greedy on the Values, not on the costs**. At each state, we compare the options by adding the cost of the arrow to the Value of the state it leads to, and we take the smallest sum. This is still a one-step-at-a-time choice, just like the greedy algorithm, but now each option already includes the whole future.

    - At A, going to B gives 12 plus 18, which is 30, and going to C gives 7 plus 24, which is 31. We go to **B**, even though its arrow is the more expensive one.
    - At B, going to D gives 5 plus 13, which is 18, going to E gives 6 plus 14, which is 20, and going to F gives 9 plus 15, which is 24. We go to **D**.
    - At D, going to G gives 8 plus 5, which is 13, going to H gives 7 plus 9, which is 16, and going to I gives 10 plus 8, which is 18. We go to **G**.
    - From G, the only move is to J.
- The best path is **A, B, D, G, J**, and its total is 12 plus 5 plus 8 plus 5, which equals 30. This matches V_{AJ} exactly.
- **Dynamic programming** is this two-part method. First, calculate the Values by working backward from the goal. Second, choose moves greedily using those Values. The method needed only one small minimum calculation per state instead of adding up all 18 routes one by one.

Section 8: Dynamic Programming Is Foresight

- **Dynamic programming is foresight**. Consider standing at the third stage, at D, E, or F. Greedy reasoning based only on arrow costs would often be pulled toward H, since the arrow into H costs 7 from every one of D, E, and F, which is the cheapest or tied for cheapest at each of them. Dynamic programming reveals that **state H leads to something terrible**: the arrow out of H into J costs 9, the largest final cost in the network, so H has the largest Value in its stage.
    - At E, for example, the arrow cost alone favors H at 7 over G at 9. Adding the Values changes the answer. Going to H gives 7 plus 9, which is 16, while going to G gives 9 plus 5, which is 14. The Value of H warns us about the 9 before we ever reach it.
- This warning still works if there are many more stages between the current state and J. However far away the bad cost lies, it has already been folded into the Values of every state that leads toward it.
- In the original problem, a **late obstacle**, the 9 on the arrow from H to J, caused an early greedy decision based on arrow cost to lead to a bad path. The greedy algorithm picked C at the first step and ended up paying for that 9 at the last step.
- **The Values contain the future**. Each Value already includes information about every obstacle between that state and the goal, so we can safely go greedy on them. The Values make **present decisions optimal**, since the whole future from each state is encapsulated in its Value.
- **Søren Kierkegaard** wrote that life can only be understood backwards, but it must be lived forwards. Dynamic programming has exactly this structure. The Values are understood backwards, calculated starting from the goal and moving toward the start. The decisions are lived forwards, made one at a time starting from A and moving toward J.

Section 9: Dynamic Programming as Decomposition into Trivial Problems

- Dynamic programming starts by posing a problem, then its subproblems, then its sub-subproblems, and so on. The big problem is solved by solving the subproblems, and the subproblems are solved by solving the sub-subproblems.
- In our network, the big problem is finding V_{AJ}. Its subproblems are finding V_{BJ} and V_{CJ}. Their subproblems are finding V_{DJ}, V_{EJ}, and V_{FJ}. Their subproblems are finding V_{GJ}, V_{HJ}, and V_{IJ}.
- The breakdown continues until we reach a point where the problems are **trivial**, meaning they can be answered with no calculation at all. For G, H, and I, the answer is simply the cost of the one arrow to J.
- Once the trivial problems are solved, we go back and solve the bigger problems, one level at a time, until we arrive at the original problem.

Section 10: Propagating Values Backward with Bellman's Rule

- The bigger problems are solved by **propagating Values** from each stage to the earlier stage using Bellman's idea. To propagate means to pass information along, and here the information moves from later stages to earlier ones.

- **Bellman's rule** states that the Value at one state is the minimum of a set of numbers. Each number in the set is the sum of the cost to one of the next states and the Value at that next state. Every equation from V_{AJ} down to V_{DJ}, V_{EJ}, and V_{FJ} follows this same pattern.
- The rule is applied the same way at every stage.
  - The rule starts with the stage just before the goal, where the Values are known directly.
  - For each state in the stage before that, we list every arrow leaving it, add each arrow's cost to the Value of the state where that arrow ends, and keep the smallest sum as that state's Value.
  - Once a whole stage has Values, the same step is repeated for the stage before it.
  - The process stops when the starting state has its Value.
- Each state needs only a handful of additions and one comparison, and each Value is calculated once and reused by every earlier state that connects to it. For example, $V_{DJ} = 13$ is calculated once and then used in both the B equation and the C equation. This reuse is what keeps dynamic programming efficient when there are many stages and states.

Section 11: Going Through the Bad 12

- Dynamic programming tells us to **go through the bad 12 earlier** to get onto a route that does not get stuck later at the 9. Paying 12 to reach B instead of 7 to reach C looks like a mistake at the first step, but it puts us on the route with the lowest total. It may seem that making such a choice requires a big brain, since the decision-maker has to weigh the whole future against the present.
- The same trade-off appears in the problem of a lifeguard reaching a person.
  - When the lifeguard and the person are both on sand, the lifeguard runs at a **constant speed** the entire way. The fastest route is then the **shortest path**, a straight line from the lifeguard to the person. This holds whether the person is directly across the sand or off at an angle along the beach.
  - When the person is in the water, the speeds are **different**. The lifeguard runs fast on sand but swims slowly in water. The **path of least time** is no longer a straight line. The lifeguard runs a longer distance along the sand, reaching the water at a point closer to the swimmer, and then swims a short distance. The route bends at the water's edge.
- The lifeguard's best route accepts a longer stretch of running, which looks worse by distance, to cut down the slow and costly swimming later. This is the same structure as paying 12 early to avoid the 9 at the end.

Section 12: Decision-Making Is Like Physics

- **Decision-making is like physics**. Light shows the same behavior as the lifeguard, in the case of **refraction**, which is the bending of light as it passes from one material into another.
  - Light travels **faster in air** and **slower in water**.
  - When light passes between air and water, its path bends at the surface, just as the lifeguard's route bends at the water's edge.
  - This bending is why a pencil standing in a glass of water appears broken or bent at the water line. The light coming from the underwater part of the pencil changes direction at the surface before reaching the eye.
- Light takes a path shaped the way the lifeguard's best route is shaped, with a bend at the boundary between a fast region and a slow region. Light has no brain at all, so choosing a route that accepts an early cost to avoid a larger later one does not require a big brain. Physical systems arrive at these optimal paths on their own, and dynamic programming describes the same kind of optimal route through a series of stages.

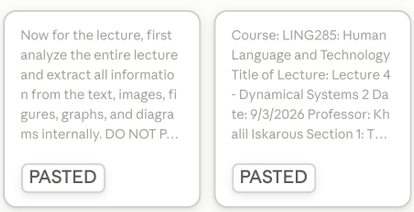
Section 13: Figuring Out the Best Route

- The complete solution can be written directly on the network by labeling each state with two things: its **Value** and the **best next state** to move to.
  - A is labeled 30 and B. The best cost from A is 30, and the best move is to B.
  - B is labeled 18 and D. C is labeled 24 and E.

    - D is labeled 13 and G. E is labeled 14 and G. F is labeled 15 and G.
    - G, H, and I are labeled 5 and J, 9 and J, and 8 and J.
- With these labels, the best route can be read off by starting at A and following each state's best next state: A to B, B to D, D to G, and G to J, for a total of 30.
- The labels also give the best route from **any** state, not just from A. If we somehow ended up at C, the labels tell us to go to E, then G, then J, and that the cost from C will be 24. A single backward calculation solves the problem for every possible starting point at once.

Section 14: Pop Quiz — Systems That Settle to the Same Value

- A learning process can be tracked by plotting a quantity x against **epochs**, where an epoch is one round of learning in which the value of x gets updated. The **learning rate** controls how large each update is, and here it is set to **10%**.
- Three runs of the process start from different **initial conditions**, abbreviated **IC**, which are the starting values of x.
    - The run with $\text{IC} = 130$ starts high and drops quickly toward about 80.
    - The run with $\text{IC} = 10$ starts low and rises quickly toward about 80.
    - The run with $\text{IC} = 80$ starts at about 80 and stays there.
    - Within roughly the first 25 to 50 epochs, all three runs meet, and from then until epoch 300 they fluctuate together in a narrow band around 80.
- The final value does not depend on where the process starts. The initial condition only changes the early part of the curve, while the long-run behavior is the same for all three runs. The changes are fastest at the beginning, when x is farthest from 80, and they slow down as x gets closer.
- Coffee changing temperature in a room follows the same pattern, tracked as temperature in °C against time in minutes over 8 minutes, with **room temperature at 18°C**.
    - Three cups of **hot coffee** start at 100°C, 80°C, and 60°C. All three cool toward room temperature, and by 8 minutes they are at roughly 29°C, 26°C, and 23°C.
    - Three cups of **iced coffee** start at 10°C, 5°C, and 0°C. All three warm toward room temperature, and by 8 minutes they are all close to it, at roughly 16°C to 17°C.
    - The hot coffee starting at 100°C drops much faster at first than the one starting at 60°C. The larger the gap between the coffee and the room, the faster the temperature changes, and the change slows down as the gap shrinks.
- In both systems, many different starting values move toward the same final value: about 80 for the learning process and 18°C for the coffee. The speed of change depends on how far the current value is from that final value.



Course: LING285: Human Language and Technology

Title of Lecture: Lecture 4 - Dynamical Systems 2

Date: 9/3/2026

Professor: Khalil Iskarous

Section 1: The Shortest Path Problem

- The **shortest path problem** asks for the cheapest route through a network from a starting point to an ending point. Each point is a **state**, each arrow is a possible move between states, and each arrow carries a **cost**. The total cost of a route is the sum of the costs on its arrows, and the goal is the route with the smallest total.
- The network starts at **A** and ends at **J**. The states sit in columns called **stages**, and every arrow moves forward exactly one stage.

    - Stage one is A. Stage two is B and C. Stage three is D, E, and F. Stage four is G, H, and I. Stage five is J.
    - Every route from A to J crosses four arrows, so it involves four decisions.
- The arrow costs out of the first two stages are:
    - From A, 12 to B and 7 to C.
    - From B, 5 to D, 6 to E, and 9 to F.
    - From C, 14 to D, 10 to E, and 11 to F.
- The arrow costs out of the last two stages are:
    - From D, 8 to G, 7 to H, and 10 to I.
    - From E, 9 to G, 7 to H, and 9 to I.
    - From F, 10 to G, 7 to H, and 8 to I.
    - From G, H, and I, the only arrows go to J, costing 5, 9, and 8.
- Each choice has an immediate cost, and it also decides which states, and therefore which future costs, are available next. A choice that is cheap now can lead into expensive moves later.

Section 2: The Greedy Algorithm

- A **greedy algorithm** chooses at every state the cheapest arrow leaving it. It never looks past the next move.
- Greed produces the route A, C, E, H, J.
    - At A, 7 to C beats 12 to B.
    - At C, 10 to E beats 14 to D and 11 to F.
    - At E, 7 to H beats 9 to G and 9 to I.
    - At H, the only move is 9 to J.
- The greedy total is:

$$7 + 10 + 7 + 9 = 33$$

- The route A, B, D, G, J starts with the more expensive move of 12 but costs less overall:

$$12 + 5 + 8 + 5 = 30$$

- **Greed is not good** here. The cheap first move to C leads into more costly continuations, and greed walks into H, which ends with the large final cost of 9. The greedy algorithm cannot see later costs, so a late obstacle can turn an early cheap choice into a bad one.

Section 3: A Small Example of Greed Failing

- In a small network, two routes lead from a start to an end, each through one middle state.
    - The top route costs 10 for the first arrow and 10 for the second.
    - The bottom route costs 2 for the first arrow and 100 for the second.
- Greed compares 10 with 2, takes the bottom route, and then must pay 100. The totals are:

$$10 + 10 = 20 \qquad 2 + 100 = 102$$

- Inside each middle state is the cost of getting from that state to the end: 10 for the top state and 100 for the bottom state. Adding each first-arrow cost to the number stored in the state it reaches gives 20 versus 102, which makes the correct choice immediate.

Section 4: The Exhaustive Algorithm

- An **exhaustive algorithm** lists every possible route, adds up each one's cost, and picks the smallest total. It checks everything, so it is **very accurate**.
- The number of routes is the product of the number of choices at each stage:

$$2 \times 3 \times 3 \times 1 = 18$$

- Each of the 18 routes has 4 arrows, so each needs 3 additions, which is 54 additions in total before comparing the 18 totals.

- Each added stage multiplies the number of routes again, and more states per stage make each multiplier larger. With **many stages and states**, exhaustive search takes a very, very long time.

Section 5: Bellman's Idea — Imagining the Values

- **Bellman** began by imagining that we already know the best cost from B to J and from C to J, written V_{BJ} and V_{CJ}. The **Value** of a state is the lowest total cost of getting from that state to the goal.
- With those Values, the best cost from A, V_{AJ}, follows from one comparison.
  - Every route from A goes first to B or C.
  - Going to B costs 12, and the best possible remaining cost is V_{BJ}, giving $12 + V_{BJ}$.
  - Going to C costs 7, and the best possible remaining cost is V_{CJ}, giving $7 + V_{CJ}$.
  - The best cost from A is the smaller of the two:

$$V_{AJ} = \min \left\{ \begin{array}{l} 12 + V_{BJ} \\ 7 + V_{CJ} \end{array} \right\}$$

- **min** means the smallest number in the set. Each entry is an arrow cost plus the Value of the state that arrow reaches.
- We **don't know** V_{BJ} or V_{CJ}. Bellman's strategy is to **decompose a problem into subproblems you can solve**, "if you can." Finding V_{AJ} becomes finding V_{BJ} and V_{CJ}, and any subproblem that still cannot be solved is broken down again.

Section 6: Breaking the Problem into Subproblems

- We don't know the best cost from B to J, but if we knew the best costs from D, E, and F to J, we could find it. The same holds for C:

$$V_{BJ} = \min \left\{ \begin{array}{l} 5 + V_{DJ} \\ 6 + V_{EJ} \\ 9 + V_{FJ} \end{array} \right\} \qquad V_{CJ} = \min \left\{ \begin{array}{l} 14 + V_{DJ} \\ 10 + V_{EJ} \\ 11 + V_{FJ} \end{array} \right\}$$

- The Values of D, E, and F depend in turn on the Values of G, H, and I:

$$V_{DJ} = \min \left\{ \begin{array}{l} 8 + V_{GJ} \\ 7 + V_{HJ} \\ 10 + V_{IJ} \end{array} \right\} \qquad V_{EJ} = \min \left\{ \begin{array}{l} 9 + V_{GJ} \\ 7 + V_{HJ} \\ 9 + V_{IJ} \end{array} \right\} \qquad V_{FJ} = \min \left\{ \begin{array}{l} 10 + V_{GJ} \\ 7 + V_{HJ} \\ 8 + V_{IJ} \end{array} \right\}$$

- G, H, and I each have one arrow straight to J, so there is nothing to decide and each Value is that arrow's cost:

$$V_{GJ} = 5 \qquad V_{HJ} = 9 \qquad V_{IJ} = 8$$

- The known Values of G, H, and I feed into the equations for D, E, and F. Those results feed into the equations for B and C, and those feed into the equation for A. Information about reaching J moves backward through the network one stage at a time.

Section 7: Calculating the Values, Then Going Greedy

- Substituting the fourth-stage Values gives the third-stage Values:

$$V_{DJ} = \min \left\{ \begin{array}{l} 8 + 5 \\ 7 + 9 \\ 10 + 8 \end{array} \right\} = \min \left\{ \begin{array}{l} 13 \\ 16 \\ 18 \end{array} \right\} = 13$$

$$V_{EJ} = \min \left\{ \begin{array}{l} 9 + 5 \\ 7 + 9 \\ 9 + 8 \end{array} \right\} = \min \left\{ \begin{array}{l} 14 \\ 16 \\ 17 \end{array} \right\} = 14$$

$$V_{FJ} = \min \left\{ \begin{array}{l} 10 + 5 \\ 7 + 9 \\ 8 + 8 \end{array} \right\} = \min \left\{ \begin{array}{l} 15 \\ 16 \\ 16 \end{array} \right\} = 15$$

- In all three, the smallest total comes from going to G, whose Value of 5 is the lowest in its stage.
- Substituting the third-stage Values gives the second-stage Values:

$$V_{BJ} = \min \left\{ \begin{array}{c} 5+13 \\ 6+14 \\ 9+15 \end{array} \right\} = \min \left\{ \begin{array}{c} 18 \\ 20 \\ 24 \end{array} \right\} = 18$$

$$V_{CJ} = \min \left\{ \begin{array}{c} 14+13 \\ 10+14 \\ 11+15 \end{array} \right\} = \min \left\{ \begin{array}{c} 27 \\ 24 \\ 26 \end{array} \right\} = 24$$

- Substituting these gives the Value of A:

$$V_{AJ} = \min \left\{ \begin{array}{c} 12+18 \\ 7+24 \end{array} \right\} = \min \left\{ \begin{array}{c} 30 \\ 31 \end{array} \right\} = 30$$

- The Values are A 30, B 18, C 24, D 13, E 14, F 15, G 5, H 9, and I 8.
- The best path comes from **going greedy on the Values, not on the costs**. At each state, add each arrow's cost to the Value of the state it reaches and take the smallest sum.
  - At A, 12 plus 18 is 30 and 7 plus 24 is 31, so go to **B**, even though its arrow costs more.
  - At B, the sums are 18, 20, and 24, so go to **D**.
  - At D, the sums are 13, 16, and 18, so go to **G**, and from G go to J.
- The best path is **A, B, D, G, J**, with total cost 30, matching V_{AJ}. **Dynamic programming** calculates the Values first and then chooses greedily using them.

Section 8: Dynamic Programming Is Foresight

- **Dynamic programming is foresight**. At the third stage, the arrow into H costs 7 from each of D, E, and F, which is the cheapest or tied for cheapest every time. The Values reveal that **H leads to something terrible**: its arrow to J costs 9, the largest final cost, so H has the largest Value in its stage.
  - At E, arrow cost alone favors H at 7 over G at 9. With Values, H gives 7 plus 9, which is 16, and G gives 9 plus 5, which is 14, so G is better.
- This remains true even if there are many more stages before J, since any distant cost is already included in the Values of the states leading to it.
- In the original problem, the **late obstacle** of 9 caused the early greedy choice based on cost to lead to a bad path. **The Values contain the future**: each one includes every obstacle between that state and the goal. Going greedy on Values therefore makes **present decisions optimal**, since the whole future from each state is encapsulated in its Value.
- **Søren Kierkegaard** wrote that life can only be understood backwards, but it must be lived forwards. In dynamic programming, the Values are calculated backward from the goal, and the decisions are made forward from the start.

Section 9: Dynamic Programming as Decomposition into Trivial Problems

- Dynamic programming poses a problem, then its subproblems, then its sub-subproblems. The big problem is solved by solving the subproblems, which are solved by solving the sub-subproblems.
  - Finding V_{AJ} requires V_{BJ} and V_{CJ}. Those require V_{DJ}, V_{EJ}, and V_{FJ}. Those require V_{GJ}, V_{HJ}, and V_{IJ}.
- The breakdown stops when the problems are **trivial**, meaning they need no calculation. For G, H, and I, the answer is the cost of the single arrow to J. The bigger problems are then solved one level at a time until the original problem is reached.

Section 10: Propagating Values Backward with Bellman's Rule

- The bigger problems are solved by **propagating Values** from each stage to the earlier stage using **Bellman's rule**: the Value at a state is the minimum of a set of numbers, each of which is the cost to one of the next states plus the Value at that state.

- The rule is applied stage by stage.
  - Start with the stage just before the goal, where the Values are known directly.
  - For each state in the previous stage, add each outgoing arrow's cost to the Value of the state it reaches, and keep the smallest sum as that state's Value.
  - Repeat for each earlier stage until the starting state has its Value.
- Each Value is calculated once and reused by every earlier state connected to it. For example, $V_{DJ} = 13$ is used in both the B equation and the C equation, which keeps the method efficient compared with checking every route.

Section 11: Going Through the Bad 12

- Dynamic programming chooses to **go through the bad 12 earlier** to reach a route that does not get stuck later at the 9. Making this kind of choice may seem to require a big brain, since it weighs the whole future against the present.
- The same choice appears in how a lifeguard reaches a person.
  - When the person is on sand, the lifeguard moves at **constant speed**, so the fastest route is the **shortest path**, a straight line. This is true whether the person is straight ahead or at an angle along the beach.
  - When the person is in the water, the speeds differ: running on sand is faster and swimming in water is slower. The **path of least time** runs a longer distance on sand and swims a shorter distance in water, so the route bends at the water's edge.
- The lifeguard accepts a longer run to cut down the slower swim, just as the best route accepts the 12 to avoid the 9.

Section 12: Decision-Making Is Like Physics

- **Decision-making is like physics**. Light behaves the same way in **refraction**, the bending of light as it moves between materials.
  - Light travels **faster in air** and **slower in water**.
  - Light's path bends at the water's surface, so a pencil standing in a glass of water appears bent at the water line when viewed from outside.
- Light has no brain, yet its path has the same shape as the lifeguard's path of least time, with a bend at the boundary between a fast region and a slow region.

Section 13: Figuring Out the Best Route

- The full solution labels each state with its **Value** and its **best next state**.
  - A is 30 and B. B is 18 and D. C is 24 and E.
  - D is 13 and G. E is 14 and G. F is 15 and G.
  - G is 5 and J, H is 9 and J, and I is 8 and J.
- Following the best next states from A gives A, B, D, G, J, with a total of 30. The labels also give the best route and its cost from any other state. From C, for example, the route is C, E, G, J, with cost 24.

Section 14: Pop Quiz — Systems That Settle to the Same Value

- A learning process tracks a quantity x over **epochs**, where each epoch is one round of updating x. The **learning rate**, which sets the size of each update, is **10%**.
- Three runs start from different **initial conditions** (IC), the starting values of x.
  - With $\mathrm{IC} = 130$, x drops quickly toward about 80.
  - With $\mathrm{IC} = 10$, x rises quickly toward about 80.
  - With $\mathrm{IC} = 80$, x stays near 80.
  - All three meet within roughly 25 to 50 epochs and then fluctuate together around 80 through epoch 300.
- The final value is the same for every starting point. The initial condition affects only the early part of the curve, and change is fastest when x is farthest from 80.
- Coffee in a room behaves the same way over 8 minutes, with **room temperature at 18°C**.
  - Hot coffee starting at 100°C, 80°C, and 60°C cools to roughly 29°C, 26°C, and 23°C by 8 minutes.

    - Iced coffee starting at 10°C, 5°C, and 0°C warms to roughly 16°C to 17°C by 8 minutes.
    - The coffee starting at 100°C cools much faster at first than the one starting at 60°C. The larger the gap between the coffee and the room, the faster the temperature changes, and the change slows as the gap shrinks.
- In both systems, different starting values move toward the same final value: about 80 for the learning process and 18°C for the coffee. The speed of change depends on the distance from that final value.